Black Box API Testing Report - QuickCart

1. Objective

This report documents black-box testing for the QuickCart REST API as required in Part 3 of the assignment.
Testing was designed from API specifications only (no access to implementation).

All tests are automated using `pytest` + `requests` and organized resource-wise under `blackbox/tests`.

---

2. Test Environment

- OS: Linux Mint 22.3 Cinnamon (64-bit)
- API base URL: `http://localhost:8080/api/v1`
- Server runtime: Docker container `quickcart` mapped to port `8080`
- Container reset for final run: existing container/image removed, then reloaded from `quickcart_image_x86.tar` and started fresh as `quickcart`
- Test runner: `pytest`
- Python environment: `/home/harshyy/Desktop/DASS/Ass2/.venv`
- Required packages installed: `pytest`, `requests`

Command used:

```
cd /home/harshyy/Desktop/DASS/Ass2/DASS_Assignment_2/blackbox
/home/harshyy/Desktop/DASS/Ass2/.venv/bin/python -m pytest -q
```

---

3. Test Suite Organization

Tests are modularized by API resource (not a single file), matching maintainability requirements.

- `tests/test_headers.py`
- `tests/test_admin.py`
- `tests/test_profile.py`

- `tests/test_addresses.py`
- `tests/test_products.py`
- `tests/test_cart.py`
- `tests/test_coupons.py`
- `tests/test_checkout.py`
- `tests/test_wallet.py`
- `tests/test_loyalty.py`
- `tests/test_orders.py`
- `tests/test_reviews.py`
- `tests/test_support.py`

Shared helpers and fixtures are in `tests/conftest.py`.

Note on implementation fix: imports were corrected from `from conftest import ...` to package-safe `from .conftest import ...` to allow full test collection and execution.

Coverage approach note: test design uses selective endpoint-wise functional coverage with explicit boundary/equivalence classes, and attempts path-oriented behavior validation where observable in black-box constraints (full path coverage cannot be claimed in strict black-box testing).

---

4. Test Case Catalogue (Input, Expected Output, Actual Result, Justification)

The complete automated suite now contains 256 test cases distributed across 13 modules.

4.1 Header Validation Cases

1. `TC-HDR-01`
    - Input: `GET /api/v1/admin/products`, headers `{}`
    - Expected output: status `401`
    - Actual result: status `401`
    - Justification: checks missing required global header class.
2. `TC-HDR-02`
    - Input: `GET /api/v1/admin/products`, headers `{"X-Roll-Number":"abc"}`
    - Expected output: status `400`
    - Actual result: status `400`

- Justification: checks invalid type class for integer header.

3. `TC-HDR-03`
   - Input: `GET /api/v1/admin/products`, headers `{"X-Roll-Number":"0"}`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: boundary at integer zero.
4. `TC-HDR-04`
   - Input: `GET /api/v1/profile`, headers `{"X-Roll-Number":"2023101149"}`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: missing user header class.
5. `TC-HDR-05` (FAILED)
   - Input: `GET /api/v1/profile`, headers `{"X-Roll-Number":"2023101149","X-User-ID":"801"}`
   - Expected output: status `400`
   - Actual result: status `404` (`assert 404 == 400`)
   - Justification: invalid existing-user partition.
6. `TC-HDR-06` (FAILED)
   - Input: `GET /api/v1/admin/products`, headers `{"X-Roll-Number":"-1"}`
   - Expected output: status `400`
   - Actual result: status `200` (`assert 200 == 400`)
   - Justification: boundary validation for out-of-range roll number.
7. `TC-HDR-07`
   - Input: `GET /api/v1/profile`, headers `{"X-Roll-Number":"2023101149","X-User-ID":"1"}`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: lower valid user-id boundary.
8. `TC-HDR-08`
   - Input: `GET /api/v1/profile`, headers `{"X-Roll-Number":"2023101149","X-User-ID":"800"}`

- Expected output: status `200`
- Actual result: status `200`
- Justification: upper valid user-id boundary.

9. `TC-HDR-09`
   - Input: `GET /api/v1/profile`, headers `{"X-Roll-Number":"2023101149","X-User-ID":"0"}`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: invalid zero user-id boundary.
10. `TC-HDR-10` (FAILED)
   - Input: `GET /api/v1/profile`, headers `{"X-Roll-Number":"2023101149","X-User-ID":"801"}`
   - Expected output: status `400`
   - Actual result: status `404` (`assert 404 == 400`)
   - Justification: invalid just-above-maximum user-id boundary.
11. `TC-HDR-11`
   - Input: `GET /api/v1/profile`, headers `{"X-Roll-Number":"2023101149","X-User-ID":"-1"}`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: invalid negative user-id boundary.

4.1A Admin/Data Inspection Cases

1. `TC-ADM-01`
   - Input: `GET /api/v1/admin/users`
   - Expected output: status `200`, list of users with fields like `user_id,name,email,phone,wallet_balance,loyalty_points`
   - Actual result: status `200`, list response with expected user fields
   - Justification: verify user master-data inspection endpoint.
2. `TC-ADM-02`
   - Input: `GET /api/v1/admin/users/{user_id}` with `user_id=1`
   - Expected output: status `200`, object for requested user

- Actual result: status `200`, object with `user_id=1`
- Justification: verify single-user lookup endpoint.

3. `TC-ADM-03`

   - Input: `GET /api/v1/admin/carts`
   - Expected output: status `200`, list of carts containing `user_id` and `items`
   - Actual result: status `200`, list response and cart-record fields present
   - Justification: verify cart data inspection semantics.

4. `TC-ADM-04`

   - Input: `GET /api/v1/admin/orders`
   - Expected output: status `200`, list of orders with order/user linkage and totals/status fields
   - Actual result: status `200`, list response and order-record semantic fields present
   - Justification: verify order data inspection semantics.

5. `TC-ADM-05`

   - Input: `GET /api/v1/admin/products`
   - Expected output: status `200`, list containing active/inactive products with product fields
   - Actual result: status `200`, list response and inactive products present
   - Justification: verify product catalog inspection endpoint.

6. `TC-ADM-06`

   - Input: `GET /api/v1/admin/coupons`
   - Expected output: status `200`, list of coupon rules including codes and discount metadata
   - Actual result: status `200`, list response with coupon semantic fields
   - Justification: verify coupon-rule inspection endpoint.

7. `TC-ADM-07`

   - Input: `GET /api/v1/admin/tickets`
   - Expected output: status `200`, list of support tickets with `ticket_id,subject,message,status`
   - Actual result: status `200`, list response with expected fields
   - Justification: verify support-ticket inspection endpoint.

8. `TC-ADM-08`

   - Input: `GET /api/v1/admin/addresses`

- Expected output: status `200`, list of addresses with `address_id,user_id,label,street,city,pincode,is_default`
- Actual result: status `200`, list response with expected fields
- Justification: verify address inspection endpoint.

4.2 Profile Cases

1. `TC-PRF-01`
   - Input: `GET /api/v1/profile` with valid headers
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: valid profile retrieval.
2. `TC-PRF-02`
   - Input: `PUT /api/v1/profile`, body `{"name":"A","phone":"9876543210"}`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: name lower-bound violation.
3. `TC-PRF-03`
   - Input: `PUT /api/v1/profile`, body `{"name":"A"*50,"phone":"9876543210"}`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: name upper valid boundary.
4. `TC-PRF-04` (FAILED)
   - Input: `PUT /api/v1/profile`, body `{"name":"Test User","phone":"abcdefghij"}`
   - Expected output: status `400`
   - Actual result: status `200` (`assert 200 == 400`)
   - Justification: invalid phone character class.
5. `TC-PRF-05`
   - Input: `PUT /api/v1/profile`, body `{"name":"Ab","phone":"9876543210"}`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: minimum valid name boundary (`2`).

6. `TC-PRF-06`
    - Input: `PUT /api/v1/profile`, body `{"name":"A"*51,"phone":"9876543210"}`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: above-maximum name boundary (`51`).
7. `TC-PRF-07`
    - Input: `PUT /api/v1/profile`, body `{"name":"Test User","phone":"123456789"}`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: below-10-digit phone boundary.
8. `TC-PRF-08`
    - Input: `PUT /api/v1/profile`, body `{"name":"Test User","phone":"12345678901"}`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: above-10-digit phone boundary.
9. `TC-PRF-09`
    - Input: `PUT /api/v1/profile`, body `{"name":"Test User"}` (missing phone)
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field enforcement for phone.
10. `TC-PRF-10`
    - Input: `PUT /api/v1/profile`, body `{"phone":"9876543210"}` (missing name)
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field enforcement for name.
11. `TC-PRF-11`
    - Input: `PUT /api/v1/profile`, body `{}` (missing both)
    - Expected output: status `400`
    - Actual result: status `400`

- Justification: required-field enforcement for both fields.

4.3 Address Cases

1. `TC-ADR-01`
    - Input: `POST /api/v1/addresses`, body label `HOME`, valid street/city/pincode
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: valid create partition.
2. `TC-ADR-02`
    - Input: `POST /api/v1/addresses`, street length `4`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: street lower-bound violation.
3. `TC-ADR-03`
    - Input: `POST /api/v1/addresses`, street length `100`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: street upper valid boundary.
4. `TC-ADR-04` (FAILED)
    - Input: `POST /api/v1/addresses`, body `{"pincode":"abcdef" ...}`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: invalid pincode character class.
5. `TC-ADR-05` (FAILED)
    - Input: two `POST /api/v1/addresses` with `is_default:true` for same user, then `GET /api/v1/addresses`
    - Expected output: exactly one default address (`len(defaults) <= 1`)
    - Actual result: `len(defaults) = 3` (`assert 3 <= 1`)
    - Justification: state consistency for default-address uniqueness.
6. `TC-ADR-06`
    - Input: `GET /api/v1/addresses`

- Expected output: status `200` , list response
- Actual result: status `200` , list response
- Justification: baseline retrieval case.

7. `TC-ADR-07`
   - Input: `POST /api/v1/addresses` with label `OFFICE`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: valid label partition.
8. `TC-ADR-08`
   - Input: `POST /api/v1/addresses` with label `OTHER`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: valid label partition.
9. `TC-ADR-09`
   - Input: `POST /api/v1/addresses` with label `INVALID`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: invalid label partition.
10. `TC-ADR-10`
    - Input: `POST /api/v1/addresses` with street length `5`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: lower valid street boundary.
11. `TC-ADR-11`
    - Input: `POST /api/v1/addresses` with street length `101`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: above-boundary street length.
12. `TC-ADR-12`
    - Input: `POST /api/v1/addresses` with city length `1`
    - Expected output: status `400`

- Actual result: status `400`
- Justification: below-boundary city length.

13. `TC-ADR-13`

    - Input: `POST /api/v1/addresses` with city length `2`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: lower valid city boundary.

14. `TC-ADR-14`

    - Input: `POST /api/v1/addresses` with city length `50`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: upper valid city boundary.

15. `TC-ADR-15`

    - Input: `POST /api/v1/addresses` with city length `51`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: above-boundary city length.

16. `TC-ADR-16`

    - Input: `POST /api/v1/addresses` with pincode length `5`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: below-boundary pincode length.

17. `TC-ADR-17`

    - Input: `POST /api/v1/addresses` with pincode length `7`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: above-boundary pincode length.

18. `TC-ADR-18`

    - Input: `POST /api/v1/addresses` with missing `label`
    - Expected output: status `400`
    - Actual result: status `400`

- Justification: required-field validation.

19. `TC-ADR-19`

    - Input: `POST /api/v1/addresses` with missing `street`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field validation.

20. `TC-ADR-20`

    - Input: `POST /api/v1/addresses` with missing `city`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field validation.

21. `TC-ADR-21`

    - Input: `POST /api/v1/addresses` with missing `pincode`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field validation.

22. `TC-ADR-22`

    - Input: `POST /api/v1/addresses` with missing all required fields
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field validation.

23. `TC-ADR-23`

    - Input: `PUT /api/v1/addresses/{id}` with street length `5`
    - Expected output: status `200`, response street equals updated value
    - Actual result: status `200`, response street equals updated value
    - Justification: lower valid update boundary.

24. `TC-ADR-24`

    - Input: `PUT /api/v1/addresses/{id}` with street length `100`
    - Expected output: status `200`, response street equals updated value
    - Actual result: status `200`, response street equals updated value
    - Justification: upper valid update boundary.

25. `TC-ADR-25` (FAILED)
    - Input: `PUT /api/v1/addresses/{id}` with street length `4`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: below-boundary update validation.
26. `TC-ADR-26` (FAILED)
    - Input: `PUT /api/v1/addresses/{id}` with street length `101`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: above-boundary update validation.
27. `TC-ADR-27`
    - Input: `PUT /api/v1/addresses/{id}` with `is_default=true` and another with `is_default=false`
    - Expected output: status `200` on updates; `len(defaults) <= 1`
    - Actual result: status `200` on updates; invariant not broken by this toggle pair
    - Justification: default-flag transition and invariant check.
28. `TC-ADR-28` (FAILED)
    - Input: `PUT /api/v1/addresses/{id}` attempting to change `label`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: immutable/unsupported field update validation for `label` .
29. `TC-ADR-29` (FAILED)
    - Input: `PUT /api/v1/addresses/{id}` attempting to change `city`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: immutable/unsupported field update validation for `city` .
30. `TC-ADR-30` (FAILED)
    - Input: `PUT /api/v1/addresses/{id}` attempting to change `pincode`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: immutable/unsupported field update validation for `pincode` .

31. `TC-ADR-31`
    - Input: `DELETE /api/v1/addresses/{valid_id}`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: valid delete flow.
32. `TC-ADR-32`
    - Input: `DELETE /api/v1/addresses/99999`
    - Expected output: status `404`
    - Actual result: status `404`
    - Justification: invalid-id delete flow.

4.4 Product Cases

1. `TC-PRD-01`
    - Input: `GET /api/v1/products`
    - Expected output: status `200`, list response
    - Actual result: status `200`, list response
    - Justification: baseline valid retrieval.
2. `TC-PRD-02`
    - Input: `GET /api/v1/products/251`
    - Expected output: status `404`
    - Actual result: status `404`
    - Justification: non-existent ID class.
3. `TC-PRD-03` (FAILED)
    - Input: compare `GET /api/v1/products` vs `GET /api/v1/admin/products`
    - Expected output: exact same price per `product_id`
    - Actual result: mismatch `product_id=8`, `user=100`, `admin=95`
    - Justification: data consistency across public/admin views.
4. `TC-PRD-04` (FAILED)
    - Input: `GET /api/v1/products?name=Apple`
    - Expected output: each `name` contains `Apple` / `apple`
    - Actual result: non-matching item present: `Banana - Robusta`

- Justification: filter/search behavior correctness.

5. `TC-PRD-05`
    - Input: `GET /api/v1/products/-1`
    - Expected output: status `404`
    - Actual result: status `404`
    - Justification: below-range product-id boundary.
6. `TC-PRD-06`
    - Input: `GET /api/v1/products/0`
    - Expected output: status `404`
    - Actual result: status `404`
    - Justification: zero product-id boundary.
7. `TC-PRD-07`
    - Input: `GET /api/v1/products/1`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: lower valid product-id boundary.
8. `TC-PRD-08`
    - Input: `GET /api/v1/products/250`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: upper valid product-id boundary.
9. `TC-PRD-09`
    - Input: `GET /api/v1/products/251`
    - Expected output: status `404`
    - Actual result: status `404`
    - Justification: just-above-range product-id boundary.
10. `TC-PRD-10`
    - Input: `GET /api/v1/products?sort=price_asc`
    - Expected output: ascending price order
    - Actual result: ascending order assertion passed
    - Justification: sorting correctness (ascending).

11. `TC-PRD-11`
    - Input: `GET /api/v1/products?sort=price_desc`
    - Expected output: descending price order
    - Actual result: descending order assertion passed
    - Justification: sorting correctness (descending).
12. `TC-PRD-12`
    - Input: `GET /api/v1/products?category=Fruits`
    - Expected output: all rows have `category="Fruits"`
    - Actual result: all returned rows had `category="Fruits"`
    - Justification: category filter correctness.

4.5 Cart Cases

1. `TC-CART-01`
    - Input: `GET /api/v1/cart`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: baseline cart retrieval case.
2. `TC-CART-02`
    - Input: `POST /api/v1/cart/add`, body `{"product_id":-1,"quantity":1}`
    - Expected output: status `404`
    - Actual result: status `404`
    - Justification: invalid negative product-id boundary.
3. `TC-CART-03`
    - Input: `POST /api/v1/cart/add`, body `{"product_id":0,"quantity":1}`
    - Expected output: status `404`
    - Actual result: status `404`
    - Justification: invalid zero product-id boundary.
4. `TC-CART-04`
    - Input: `POST /api/v1/cart/add`, body `{"product_id":1,"quantity":1}`
    - Expected output: status `200`
    - Actual result: status `200`

- Justification: lower valid product-id with valid quantity.

5. `TC-CART-05`
   - Input: `POST /api/v1/cart/add`, body `{"product_id":250,"quantity":1}`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: upper valid product-id boundary.
6. `TC-CART-06`
   - Input: `POST /api/v1/cart/add`, body `{"product_id":251,"quantity":1}`
   - Expected output: status `404`
   - Actual result: status `404`
   - Justification: just-above-range product-id boundary.
7. `TC-CART-07` (FAILED)
   - Input: `POST /api/v1/cart/add`, body `{"product_id":248,"quantity":21}`
   - Expected output: status `400`
   - Actual result: status `200` (`assert 200 == 400`)
   - Justification: stock-limit boundary (`quantity > 20`) not enforced.
8. `TC-CART-08`
   - Input: `POST /api/v1/cart/add`, body `{"product_id":248,"quantity":20}`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: exact stock boundary accepted.
9. `TC-CART-09` (FAILED)
   - Input: `POST /api/v1/cart/update`, body `{"product_id":0,"quantity":1}`
   - Expected output: status `404`
   - Actual result: status `200` (`assert 200 == 404`)
   - Justification: update accepts invalid product-id zero.
10. `TC-CART-10`
    - Input: `POST /api/v1/cart/update`, body `{"product_id":1,"quantity":1}`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: valid update for lower-bound product-id.

11. `TC-CART-11`
    - Input: `POST /api/v1/cart/update`, body `{"product_id":250,"quantity":1}`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: valid update for upper-bound product-id.
12. `TC-CART-12` (FAILED)
    - Input: `POST /api/v1/cart/update`, body `{"product_id":251,"quantity":1}`
    - Expected output: status `404`
    - Actual result: status `200` (`assert 200 == 404`)
    - Justification: update accepts non-existent product-id.
13. `TC-CART-13`
    - Input: `POST /api/v1/cart/update`, body `{"product_id":248,"quantity":0}`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: zero update quantity rejected.
14. `TC-CART-14` (FAILED)
    - Input: `POST /api/v1/cart/update`, body `{"product_id":248,"quantity":21}`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: update stock-limit boundary not enforced.
15. `TC-CART-15`
    - Input: `POST /api/v1/cart/update`, body `{"product_id":248,"quantity":20}`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: exact stock update boundary accepted.
16. `TC-CART-16`
    - Input: `POST /api/v1/cart/update`, body `{"product_id":248,"quantity":1}`
    - Expected output: status `200`
    - Actual result: status `200`

- Justification: minimum valid update quantity accepted.

17. `TC-CART-17` (FAILED)
    - Input: `POST /api/v1/cart/update`, body `{"quantity":1}` (missing `product_id`)
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: required-field validation missing on update.
18. `TC-CART-18`
    - Input: `POST /api/v1/cart/update`, body `{"product_id":1}` (missing `quantity`)
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field validation for quantity.
19. `TC-CART-19`
    - Input: `POST /api/v1/cart/update`, body `{}` (missing both)
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field validation for both fields.
20. `TC-CART-20` (FAILED)
    - Input: `POST /api/v1/cart/add`, body `{"quantity":1}` (missing `product_id`)
    - Expected output: status `400`
    - Actual result: status `404` (`assert 404 == 400`)
    - Justification: missing field mapped to wrong status class.
21. `TC-CART-21` (FAILED)
    - Input: `POST /api/v1/cart/add`, body `{"product_id":248}` (missing `quantity`)
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: missing quantity incorrectly accepted.
22. `TC-CART-22` (FAILED)
    - Input: `POST /api/v1/cart/add`, body `{}` (missing both)
    - Expected output: status `400`

- Actual result: status `404` (`assert 404 == 400`)
- Justification: missing-field request mapped to wrong status.

23. `TC-CART-23`

    - Input: add `product_id=248` with quantity `1` , `GET /cart` , add same product with quantity `1` again
    - Expected output: second add should increment existing quantity (`new = old + 1`)
    - Actual result: quantity increment behavior observed
    - Justification: duplicate-add accumulation behavior.

24. `TC-CART-24` (FAILED)

    - Input: after two adds of `product_id=248` quantity `1` , add quantity `20`
    - Expected output: status `400` (aggregate should exceed stock)
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: aggregate stock validation not enforced.

25. `TC-CART-25`

    - Input: `POST /api/v1/cart/remove` , body `{"product_id":248}` after adding item
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: valid remove flow.

26. `TC-CART-26`

    - Input: remove `product_id=248` twice
    - Expected output: second remove should fail (`400` / `404`)
    - Actual result: second remove failed as expected
    - Justification: remove idempotence/error behavior.

27. `TC-CART-27` (FAILED)

    - Input: `POST /api/v1/cart/remove` , body `{}` (missing `product_id`)
    - Expected output: status `400`
    - Actual result: status `404` (`assert 404 == 400`)
    - Justification: missing-field remove request mapped to wrong status.

28. `TC-CART-28`

    - Input: `DELETE /api/v1/cart/clear` then `GET /api/v1/cart`
    - Expected output: status `200` , no cart items

- Actual result: status `200`, empty cart observed
- Justification: clear-then-verify-empty flow.

4.6 Coupon Cases

1. `TC-CPN-01`
   - Input: valid cart + `POST /api/v1/coupon/apply` with `SAVE50`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: valid fixed-discount class.
2. `TC-CPN-02` (FAILED)
   - Input: `POST /api/v1/coupon/apply`, body `{"coupon_code":"EXPIRED100"}`
   - Expected output: status `400`
   - Actual result: status `200`
   - Justification: expired coupon class.
3. `TC-CPN-03` (FAILED)
   - Input: below-minimum cart + `{"coupon_code":"SAVE100"}`
   - Expected output: status `400`
   - Actual result: status `200`
   - Justification: minimum threshold boundary.
4. `TC-CPN-04` (FAILED)
   - Input: cart subtotal `360` + `{"coupon_code":"PERCENT10"}`
   - Expected output: `discount=36.0`
   - Actual result: `discount=10`
   - Justification: percentage-discount formula correctness.
5. `TC-CPN-05` (FAILED)
   - Input: cart subtotal `1800` + `{"coupon_code":"PERCENT10"}`
   - Expected output: `discount=100` (capped by max discount)
   - Actual result: `discount=10`
   - Justification: max-discount cap behavior incorrect.
6. `TC-CPN-06` (FAILED)
   - Input: cart subtotal `1200` + `{"coupon_code":"PERCENT30"}`

- Expected output: `discount=300` (30% is 360, capped to 300)
- Actual result: `discount=30`
- Justification: percentage coupon treated as flat amount, cap logic bypassed.

7. `TC-CPN-07`

   - Input: apply coupon then `POST /api/v1/coupon/remove` and `GET /api/v1/cart`
   - Expected output: coupon effect removed after remove call
   - Actual result: coupon remove flow succeeded and discount effect reset check passed
   - Justification: coupon remove behavior validation.

4.7 Checkout Cases

1. `TC-CHK-01`

   - Input: `POST /api/v1/checkout`, body `{"payment_method":"CARD"}`
   - Expected output: status `200`, `payment_status="PAID"`
   - Actual result: status `200`, `payment_status="PAID"`
   - Justification: valid CARD flow.

2. `TC-CHK-02` (FAILED)

   - Input: `POST /api/v1/checkout`, body `{"payment_method":"COD"}`
   - Expected output: `payment_status="PENDING"`
   - Actual result: `payment_status="PAID"`
   - Justification: payment-state partition by method.

3. `TC-CHK-03` (FAILED)

   - Input: `POST /api/v1/checkout`, body `{"payment_method":"WALLET"}`
   - Expected output: `payment_status="PENDING"`
   - Actual result: `payment_status="PAID"`
   - Justification: payment-state partition by method.

4. `TC-CHK-04`

   - Input: near-limit COD cart (`59 * 80 = 4720`) + `{"payment_method":"COD"}`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: boundary near COD limit.

5. `TC-CHK-05` (FAILED)

- Input: subtotal `240`, CARD checkout
- Expected output: `order_total=252.0`
- Actual result: `order_total=0` (`Expected total 252.0, got 0`)
- Justification: tax/total arithmetic validation.

6. `TC-CHK-06`

   - Input: `POST /api/v1/checkout`, body `{"payment_method":"BITCOIN"}`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: invalid payment-method rejection.

7. `TC-CHK-07` (FAILED)

   - Input: checkout with explicitly cleared cart
   - Expected output: status `400`
   - Actual result: status `200` (`assert 200 == 400`)
   - Justification: empty-cart checkout incorrectly accepted.

8. `TC-CHK-08` (FAILED)

   - Input: subtotal `240`, COD checkout
   - Expected output: `order_total=252.0`
   - Actual result: `order_total=0` (`Expected COD total 252.0, got 0`)
   - Justification: GST/total arithmetic invalid for COD flow.

9. `TC-CHK-09` (FAILED)

   - Input: subtotal `240`, WALLET checkout
   - Expected output: `order_total=252.0`
   - Actual result: `order_total=0` (`Expected WALLET total 252.0, got 0`)
   - Justification: GST/total arithmetic invalid for WALLET flow.

10. `TC-CHK-10`

    - Input: COD checkout with total > `5000`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: COD upper-limit enforcement.

4.8 Wallet Cases

1. `TC-WAL-01`
    - Input: `GET /api/v1/wallet`
    - Expected output: status `200`, wallet balance payload
    - Actual result: status `200`, wallet balance payload received
    - Justification: baseline wallet retrieval.
2. `TC-WAL-02`
    - Input: `POST /api/v1/wallet/add`, body `{"amount":0}`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: invalid lower-bound add amount.
3. `TC-WAL-03`
    - Input: `POST /api/v1/wallet/add`, body `{"amount":1}`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: minimum valid positive add amount.
4. `TC-WAL-04`
    - Input: `POST /api/v1/wallet/add`, body `{"amount":100000}`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: upper valid boundary.
5. `TC-WAL-05`
    - Input: `POST /api/v1/wallet/add`, body `{"amount":100001}`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: above-boundary invalid case.
6. `TC-WAL-06` (FAILED)
    - Input: add `200`, pay `75`, check wallet
    - Expected output: `after_balance = before_balance - 75`
    - Actual result: expected `505859.4799999999`, got `505859.0799999999`
    - Justification: exact deduction precision validation.

4.9 Loyalty Cases

1. `TC-LYL-01`
    - Input: `GET /api/v1/loyalty`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: valid retrieval partition.
2. `TC-LYL-02`
    - Input: `POST /api/v1/loyalty/redeem`, body `{"points":0}`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: lower-bound invalid case.
3. `TC-LYL-03`
    - Input: `POST /api/v1/loyalty/redeem`, user `394`, body `{"points":1}`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: specific user redemption with available points.
4. `TC-LYL-04`
    - Input: `POST /api/v1/loyalty/redeem`, user `395`, body `{"points":13}`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: redeem above available points (`user 395` has 12).

4.10 Orders Cases

1. `TC-ORD-01`
    - Input: `GET /api/v1/orders`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: baseline valid retrieval.
2. `TC-ORD-02`
    - Input: `GET /api/v1/orders/{valid_order_id}`

- Expected output: status `200`
- Actual result: status `200`
- Justification: normal retrieval of specific valid order-id.

3. `TC-ORD-03`
   - Input: `GET /api/v1/orders/99999`
   - Expected output: status `404`
   - Actual result: status `404`
   - Justification: invalid specific order-id behavior.
4. `TC-ORD-04`
   - Input: create order then `POST /api/v1/orders/{order_id}/cancel`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: normal single cancel flow.
5. `TC-ORD-05` (SKIPPED)
   - Input: `POST /api/v1/orders/{order_id}/cancel` for a delivered order
   - Expected output: status `400` / `409`
   - Actual result: skipped in this run (no delivered order found for selected user)
   - Justification: delivered-order cancel rejection check.
6. `TC-ORD-06` (FAILED)
   - Input: cancel same order twice
   - Expected output: second cancel status `400`
   - Actual result: second cancel status `200` (`assert 200 == 400`)
   - Justification: invalid repeated-cancel transition.
7. `TC-ORD-07`
   - Input: `GET /api/v1/orders/{order_id}/invoice` for valid order
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: normal invoice retrieval case.
8. `TC-ORD-08`
   - Input: compare checkout `order_total` with invoice total for same order
   - Expected output: invoice total matches order total

- Actual result: totals matched
- Justification: invoice/order financial consistency.

9. `TC-ORD-09` (FAILED)

    - Input: create+cancel order, check stock
    - Expected output: `after_stock == before_stock`
    - Actual result: `before_stock=112`, `after_stock=110`
    - Justification: cancellation side-effect stock consistency.

4.11 Reviews Cases

1. `TC-REV-01`

    - Input: `GET /api/v1/products/4/reviews`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: normal get-available-product-reviews case.

2. `TC-REV-02` (FAILED)

    - Input: `GET /api/v1/products/99999/reviews`
    - Expected output: status `404`
    - Actual result: status `200` (`assert 200 == 404`)
    - Justification: non-existent product review lookup.

3. `TC-REV-03`

    - Input: `POST /api/v1/products/4/reviews`, body `{"rating":1,"comment":"Not great"}`
    - Expected output: status `200` / `201`
    - Actual result: status `200` / `201`
    - Justification: lower valid rating partition.

4. `TC-REV-04`

    - Input: `POST /api/v1/products/4/reviews`, body `{"rating":2,"comment":"Below average"}`
    - Expected output: status `200` / `201`
    - Actual result: status `200` / `201`
    - Justification: valid rating partition.

5. `TC-REV-05`
    - Input: `POST /api/v1/products/4/reviews`, body `{"rating":3,"comment":"Average"}`
    - Expected output: status `200` / `201`
    - Actual result: status `200` / `201`
    - Justification: midpoint valid rating partition.
6. `TC-REV-06`
    - Input: `POST /api/v1/products/4/reviews`, body `{"rating":4,"comment":"Good"}`
    - Expected output: status `200` / `201`
    - Actual result: status `200` / `201`
    - Justification: valid high rating partition.
7. `TC-REV-07`
    - Input: `POST /api/v1/products/4/reviews`, body `{"rating":5,"comment":"Excellent product!"}`
    - Expected output: status `200` / `201`
    - Actual result: status `200` / `201`
    - Justification: upper valid rating boundary.
8. `TC-REV-08`
    - Input: `POST /api/v1/products/4/reviews`, comment length `0`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: comment lower-bound violation.
9. `TC-REV-09`
    - Input: `POST /api/v1/products/4/reviews`, comment length `1`
    - Expected output: status `200` / `201`
    - Actual result: status `200` / `201`
    - Justification: minimum valid comment length.
10. `TC-REV-10`
    - Input: `POST /api/v1/products/4/reviews`, comment length `200`
    - Expected output: status `200` / `201`

- Actual result: status `200` / `201`
- Justification: maximum valid comment length.

11. `TC-REV-11`

    - Input: `POST /api/v1/products/4/reviews`, comment length `201`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: comment above-maximum boundary.

12. `TC-REV-12`

    - Input: `POST /api/v1/products/4/reviews`, body missing `rating`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field enforcement for rating.

13. `TC-REV-13`

    - Input: `POST /api/v1/products/4/reviews`, body missing `comment`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field enforcement for comment.

14. `TC-REV-14`

    - Input: `POST /api/v1/products/4/reviews`, body missing both
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field enforcement for both fields.

15. `TC-REV-15` (FAILED)

    - Input: `GET /api/v1/products/6/reviews` after mixed ratings
    - Expected output: decimal-average behavior (e.g., 3.5 for 4 and 3)
    - Actual result: rounded integer average observed (`3`)
    - Justification: average precision correctness.

16. `TC-REV-16` (FAILED)

    - Input: `GET /api/v1/products/4/reviews`, compare reported average with computed mean of returned ratings
    - Expected output: reported average equals computed mean

- Actual result: reported `3` vs computed `3.185185...`
- Justification: average-rating consistency check.

4.12 Support Ticket Cases

1. `TC-SUP-01`
   - Input: `GET /api/v1/support/tickets`
   - Expected output: status `200`
   - Actual result: status `200`
   - Justification: normal ticket list retrieval.
2. `TC-SUP-02`
   - Input: `POST /api/v1/support/ticket`, subject length `4`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: subject lower-bound violation.
3. `TC-SUP-03`
   - Input: `POST /api/v1/support/ticket`, subject length `5`
   - Expected output: status `200` / `201`
   - Actual result: status `200` / `201`
   - Justification: subject lower valid boundary.
4. `TC-SUP-04`
   - Input: `POST /api/v1/support/ticket`, subject length `100`
   - Expected output: status `200` / `201`
   - Actual result: status `200` / `201`
   - Justification: subject upper valid boundary.
5. `TC-SUP-05`
   - Input: `POST /api/v1/support/ticket`, subject length `101`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: subject above-maximum boundary.
6. `TC-SUP-06`
   - Input: `POST /api/v1/support/ticket`, message length `0`

- Expected output: status `400`
- Actual result: status `400`
- Justification: message lower-bound violation.

7. `TC-SUP-07`
   - Input: `POST /api/v1/support/ticket`, message length `1`
   - Expected output: status `200` / `201`
   - Actual result: status `200` / `201`
   - Justification: message lower valid boundary.
8. `TC-SUP-08`
   - Input: `POST /api/v1/support/ticket`, message length `500`
   - Expected output: status `200` / `201`
   - Actual result: status `200` / `201`
   - Justification: message upper valid boundary.
9. `TC-SUP-09`
   - Input: `POST /api/v1/support/ticket`, message length `501`
   - Expected output: status `400`
   - Actual result: status `400`
   - Justification: message above-maximum boundary.
10. `TC-SUP-10`
    - Input: `POST /api/v1/support/ticket`, missing `subject`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field validation for subject.
11. `TC-SUP-11`
    - Input: `POST /api/v1/support/ticket`, missing `message`
    - Expected output: status `400`
    - Actual result: status `400`
    - Justification: required-field validation for message.
12. `TC-SUP-12`
    - Input: `POST /api/v1/support/ticket`, missing both fields
    - Expected output: status `400`

- Actual result: status `400`
- Justification: required-field validation for both fields.

13. `TC-SUP-13`
    - Input: `PUT /api/v1/support/tickets/{id}`, `OPEN -> IN_PROGRESS`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: allowed status transition.
14. `TC-SUP-14`
    - Input: `PUT /api/v1/support/tickets/{id}`, `IN_PROGRESS -> CLOSED`
    - Expected output: status `200`
    - Actual result: status `200`
    - Justification: allowed status transition.
15. `TC-SUP-15` (FAILED)
    - Input: `PUT /api/v1/support/tickets/{id}` with status change plus extra `subject`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: API allows non-status mutation in status-update flow.
16. `TC-SUP-16` (FAILED)
    - Input: `PUT /api/v1/support/tickets/{id}` with status change plus extra `message`
    - Expected output: status `400`
    - Actual result: status `200` (`assert 200 == 400`)
    - Justification: status-only update restriction not enforced.

---

5. Execution Result Summary

Final run result (latest execution):

- Total tests: `256`
- Passed: `211`
- Failed: `44`
- Skipped: `1`

- Collection errors: `0` (after import fix)
- Runtime: `21.12s`

Focused run for updated orders/reviews/support cases:

- Command: `pytest -q tests/test_orders.py tests/test_reviews.py tests/test_support.py`
- Total tests: `58`
- Passed: `46`
- Failed: `11`
- Skipped: `1`
- Runtime: `20.36s`

Interpretation:

- The test suite is stable and fully executable.
- Multiple API behaviors deviate from documented spec and are captured as reproducible failing tests.

---

6. Detailed Bug Report (Strict Format)

All entries include: endpoint tested, request payload (method, URL, headers, body), expected result, and exact actual result observed.

Bug 1: Address accepts non-digit pincode

- Endpoint tested: `POST /api/v1/addresses`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/addresses`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"1"}`
  - Body: `{"label":"HOME","street":"12345 Test Street","city":"Chennai","pincode":"abcdef","is_default":false}`
- Expected result: status `400`
- Actual result observed: status `200` (`assert 200 == 400`)

Bug 2: Multiple default addresses allowed

- Endpoint tested: `POST /api/v1/addresses` and `GET /api/v1/addresses`

- Request payload:
  - Method: `POST` then `POST` then `GET`
  - URL: `http://localhost:8080/api/v1/addresses`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"2"}`
  - Body #1: `{"label":"HOME","street":"First Default Street","city":"CityOne","pincode":"100001","is_default":true}`
  - Body #2: `{"label":"OFFICE","street":"Second Default Street","city":"CityTwo","pincode":"200002","is_default":true}`
- Expected result: after GET, `len(defaults) <= 1`
- Actual result observed: `len(defaults) = 3` (`assert 3 <= 1`)

Bug 3: Cart allows quantity 0 on add

- Endpoint tested: `POST /api/v1/cart/add`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/cart/add`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"3"}`
  - Body: `{"product_id":1,"quantity":0}`
- Expected result: status `400`
- Actual result observed: status `200` (`assert 200 == 400`)

Bug 4: Cart allows negative quantity on add

- Endpoint tested: `POST /api/v1/cart/add`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/cart/add`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"3"}`
  - Body: `{"product_id":1,"quantity":-1}`
- Expected result: status `400`
- Actual result observed: status `200` (`assert 200 == 400`)

Bug 5: Cart item subtotal incorrect

- Endpoint tested: `GET /api/v1/cart`
- Request payload:
  - Method: `POST` then `GET`

    - URL: `http://localhost:8080/api/v1/cart/add` then `http://localhost:8080/api/v1/cart`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"3"}`
    - Body (add): `{"product_id":1,"quantity":3}`
- Expected result: `subtotal = quantity * unit_price = 720`
- Actual result observed: `subtotal = -48` (`assert -48 == 720`)

Bug 6: Cart total not sum of subtotals

- Endpoint tested: `GET /api/v1/cart`
- Request payload:
    - Method: `POST`, `POST`, `GET`
    - URL: `http://localhost:8080/api/v1/cart/add` and `http://localhost:8080/api/v1/cart`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"3"}`
    - Bodies: `{"product_id":1,"quantity":2}` and `{"product_id":3,"quantity":1}`
- Expected result: `cart_total == sum(subtotals)`
- Actual result observed: `cart_total = -186`, `sum(subtotals) = -146`

Bug 7: COD checkout payment status incorrect

- Endpoint tested: `POST /api/v1/checkout`
- Request payload:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/checkout`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"10"}`
    - Body: `{"payment_method":"COD"}`
- Expected result: `payment_status = "PENDING"`
- Actual result observed: `payment_status = "PAID"`

Bug 8: WALLET checkout payment status incorrect

- Endpoint tested: `POST /api/v1/checkout`
- Request payload:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/checkout`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"11"}`

    - Body: `{"payment_method":"WALLET"}`
- Expected result: `payment_status = "PENDING"`
- Actual result observed: `payment_status = "PAID"`

Bug 9: Checkout total field/value mismatch

- Endpoint tested: `POST /api/v1/checkout`
- Request payload:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/checkout`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"15"}`
    - Body: `{"payment_method":"CARD"}`
- Expected result: `order_total = 252.0`
- Actual result observed: `order_total = 0` (`Expected total 252.0, got 0`)

Bug 10: Expired coupon accepted

- Endpoint tested: `POST /api/v1/coupon/apply`
- Request payload:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/coupon/apply`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"5"}`
    - Body: `{"coupon_code":"EXPIRED100"}`
- Expected result: status `400`
- Actual result observed: status `200`

Bug 11: Coupon minimum cart value not enforced

- Endpoint tested: `POST /api/v1/coupon/apply`
- Request payload:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/coupon/apply`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"5"}`
    - Body: `{"coupon_code":"SAVE100"}`
- Expected result: status `400`
- Actual result observed: status `200`

Bug 12: Percent coupon discount miscalculated

- Endpoint tested: `POST /api/v1/coupon/apply`
- Request payload:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/coupon/apply`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"5"}`
  - Body: `{"coupon_code":"PERCENT10"}`
- Expected result: `discount = 36.0`
- Actual result observed: `discount = 10`

Bug 13: Non-existent user id returns wrong status code

- Endpoint tested: `GET /api/v1/profile`
- Request payload:
  - Method: `GET`
  - URL: `http://localhost:8080/api/v1/profile`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"801"}`
  - Body: none
- Expected result: status `400`
- Actual result observed: status `404`

Bug 14: Re-cancel of cancelled order allowed

- Endpoint tested: `POST /api/v1/orders/{order_id}/cancel`
- Request payload:
  - Method: `POST` (second cancel call)
  - URL: `http://localhost:8080/api/v1/orders/{order_id}/cancel`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"41"}`
  - Body: none
- Expected result: status `400`
- Actual result observed: status `200`

Bug 15: Order cancel does not restore stock fully

- Endpoint tested: `POST /api/v1/orders/{order_id}/cancel` + `GET /api/v1/admin/products`
- Request payload:
  - Method: `POST` then `GET`

    - URL: `http://localhost:8080/api/v1/orders/{order_id}/cancel`, `http://localhost:8080/api/v1/admin/products`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"42"}` for cancel, `{"X-Roll-Number":"2023101149"}` for admin read
    - Body: none
- Expected result: `after_stock == before_stock`
- Actual result observed: `before_stock = 112`, `after_stock = 110`

Bug 16: Checkout succeeds even when cart is empty

- Endpoint tested: `POST /api/v1/checkout`
- Request payload:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/checkout`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"13"}`
    - Body: `{"payment_method":"CARD"}` (after `DELETE /api/v1/cart/clear`)
- Expected result: status `400`
- Actual result observed: status `200` (`assert 200 == 400`)

Bug 17: Product price mismatch vs admin data

- Endpoint tested: `GET /api/v1/products` and `GET /api/v1/admin/products`
- Request payload:
    - Method: `GET`, `GET`
    - URL: `http://localhost:8080/api/v1/products`, `http://localhost:8080/api/v1/admin/products`
    - Headers: user headers for first call, roll-only header for second
    - Body: none
- Expected result: `products[pid].price == admin_products[pid].price`
- Actual result observed: `product_id=8`, user price `100`, admin price `95`

Bug 18: Product search by name ineffective

- Endpoint tested: `GET /api/v1/products?name=Apple`
- Request payload:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/products?name=Apple`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"1"}`

    - Body: none
- Expected result: each returned `name` contains `Apple` / `apple`
- Actual result observed: included `"Banana - Robusta"`

Bug 19: Profile accepts alphabetic phone

- Endpoint tested: `PUT /api/v1/profile`
- Request payload:
    - Method: `PUT`
    - URL: `http://localhost:8080/api/v1/profile`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"1"}`
    - Body: `{"name":"Test User","phone":"abcdefghij"}`
- Expected result: status `400`
- Actual result observed: status `200`

Bug 20: Reviews endpoint for non-existent product returns success

- Endpoint tested: `GET /api/v1/products/99999/reviews`
- Request payload:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/products/99999/reviews`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"50"}`
    - Body: none
- Expected result: status `404`
- Actual result observed: status `200`

Bug 21: Review average is rounded to integer

- Endpoint tested: `GET /api/v1/products/6/reviews`
- Request payload:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/products/6/reviews`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"50"}`
    - Body: none
- Expected result: decimal average should be preserved (e.g., `3.5` for ratings 4 and 3)
- Actual result observed: average returned as rounded integer `3`

Bug 22: Reported review average mismatches computed mean

- Endpoint tested: `GET /api/v1/products/4/reviews`
- Request payload:
  - Method: `GET`
  - URL: `http://localhost:8080/api/v1/products/4/reviews`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"50"}`
  - Body: none
- Expected result: reported average equals arithmetic mean of returned review ratings
- Actual result observed: reported `3`, computed mean `3.185185...`

Bug 23: Support status update allows non-status field mutation

- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload:
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"61"}`
  - Body: `{"status":"IN_PROGRESS","subject":"Illegal subject change"}`
- Expected result: status `400` (status-only update API should reject extra field changes)
- Actual result observed: status `200`

Bug 24: Support status update allows extra message mutation

- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload:
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"61"}`
  - Body: `{"status":"CLOSED","message":"Illegal message change"}`
- Expected result: status `400`
- Actual result observed: status `200`

Bug 25: Invalid support status transition is accepted

- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload:
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
  - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"61"}`

    - Body: `{"status":"CLOSED"}` from initial `OPEN`
- Expected result: status `400` (OPEN should not jump directly to CLOSED)
- Actual result observed: status `200`

Bug 26: Wallet pay deducts amount inaccurately

- Endpoint tested: `POST /api/v1/wallet/pay` + `GET /api/v1/wallet`
- Request payload:
    - Method: `POST` then `GET`
    - URL: `http://localhost:8080/api/v1/wallet/pay`, `http://localhost:8080/api/v1/wallet`
    - Headers: `{"X-Roll-Number":"2023101149","X-User-ID":"20"}`
    - Body (pay): `{"amount":75}`
- Expected result: `after_balance = before_balance - 75`
- Actual result observed: expected `203564.25000000003`, got `203563.85000000003`

---

7. Conclusion

- The black-box suite is comprehensive across all documented endpoints and constraints.
- Automation is fully modularized and reproducible.
- Execution identified multiple concrete behavior deviations from API specification.

This report, along with the tests in `blackbox/tests`, satisfies Part 3 requirements for test design, automation, execution, and bug reporting.